

技术评估复盘手册:WhatsApp 聊天后端模拟 API

练习目标: 在 1 小时内从零搭建一个基于 Python FastAPI 和 SQLite 的后端系统, 完成消息接收、关键词路由、模拟回复以及数据库读写, 并准备好最终的提交文件。

步骤一: 创建项目与环境准备

1. 新建文件夹: 在电脑上创建一个空文件夹(例如 `whatsapp-assessment`), 并将其作为你的项目根目录。
2. 打开终端: 在文件夹路径栏输入 `cmd` 并回车, 或使用 VS Code 打开该文件夹并呼出终端。
3. 安装依赖: 在终端中运行以下命令, 安装所需的 Python 库:

```
Bash
pip install fastapi uvicorn sqlalchemy pydantic
```

步骤二: 编写核心业务代码

在项目文件夹中新建一个名为 `main.py` 的文件, 并将以下完整代码粘贴进去并保存(`Ctrl + S`)。

Python

```
from fastapi import FastAPI, Depends, HTTPException, Header
from sqlalchemy import create_engine, Column, Integer, String, Text, ForeignKey
from sqlalchemy.orm import declarative_base, sessionmaker, Session
import os
```

```
# --- 1. 数据库设置 (SQLite) ---
```

```
DATABASE_URL = "sqlite:///./chat.db"
```

```
engine = create_engine(DATABASE_URL, connect_args={"check_same_thread": False})
```

```
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
```

```
Base = declarative_base()
```

```
class SessionModel(Base):
```

```
    tablename = "sessions"
```

```
    id = Column(Integer, primary_key=True, index=True)
```

```
    user_id = Column(String, index=True)
```

```
    agent = Column(String)
```

```
    last_active = Column(String)
```

```
class MessageModel(Base):
```

```

    tablename = "messages"
    id = Column(Integer, primary_key=True, index=True)
    session_id = Column(Integer, ForeignKey("sessions.id"))
    role = Column(String) # "user" 或 "assistant"
    content = Column(Text)
    event_id = Column(String, unique=True, index=True) # 防止重复的唯一约束

```

```

Base.metadata.create_all(bind=engine)

```

```

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

```

```

# --- 2. Token 身份验证 ---

```

```

def verify_token(authorization: str = Header(None)):
    expected_token = os.getenv("SECRET_TOKEN", "my secret token 123")
    if not authorization or authorization != f"Bearer {expected_token}":
        raise HTTPException(status_code=401, detail="Unauthorized")

```

```

# --- 3. 核心逻辑: 代理路由与回复生成 ---

```

```

def route_agent(message: str) -> str:
    msg_lower = message.lower()
    if any(kw in msg_lower for kw in ["buy", "price", "product"]): return "sales"
    if any(kw in msg_lower for kw in ["problem", "issue", "complaint"]): return "support"
    return "general"

```

```

def generate_reply(agent: str) -> str:
    replies = {
        "sales": "Thanks for your interest. Can you share what you are looking for?",
        "support": "Sorry you are facing this issue. Can you describe it a bit more?",
        "general": "Happy to help. What would you like to know?"
    }
    return replies.get(agent, replies["general"])

```

```

app = FastAPI()

```

```

# --- 4. API 端点 ---

```

```

@app.post("/webhook/whatsapp", dependencies=[Depends(verify_token)])
async def webhook_whatsapp(payload: dict, db: Session = Depends(get_db)):
    event_id, user_id, message = payload.get("event_id"), payload.get("user_id"),
    payload.get("message")

```

```

    # 排重检查 (Duplicate Handling)
    if db.query(MessageModel).filter(MessageModel.event_id == event_id).first():
        return {"status": "skipped", "reason": "duplicate event_id"}

```

```

# 路由与生成回复
agent_type = route_agent(message)
reply_content = generate_reply(agent_type)

# 保存 Session 数据
new_session = SessionModel(user_id=user_id, agent=agent_type, last_active="now")
db.add(new_session)
db.commit()
db.refresh(new_session)

# 保存 Messages 数据
db.add(MessageModel(session_id=new_session.id, role="user", content=message,
event_id=event_id))
db.add(MessageModel(session_id=new_session.id, role="assistant",
content=reply_content, event_id=f"reply_{event_id}"))
db.commit()

return {"status": "success", "agent": agent_type, "reply": reply_content}

@app.post("/simulate")
async def simulate(payload: dict, db: Session = Depends(get_db)):
    # 供测试用的模拟端点, 直接复用主逻辑
    return await webhook_whatsapp(payload, db)

@app.get("/sessions")
def get_sessions(db: Session = Depends(get_db)):
    return db.query(SessionModel).all()

@app.get("/sessions/{session_id}/messages")
def get_session_messages(session_id: int, db: Session = Depends(get_db)):
    return db.query(MessageModel).filter(MessageModel.session_id == session_id).all()

```

步骤三：启动服务与接口测试

1. 启动服务器：在终端中运行以下命令：
 Bash
`uvicorn main:app --reload`
 (注意：成功启动后，不要在终端里粘贴任何 JSON 代码，终端只负责保持服务器运行。)
2. 图形化接口测试：
 打开浏览器，访问
[\[http://127.0.0.1:8000/docs\]](http://127.0.0.1:8000/docs) (<http://127.0.0.1:8000/docs>)。
3. 测试数据写入 (POST /simulate):
 - 展开 POST /simulate, 点击 Try it out。

- 在 Request body 中输入测试 JSON:
 - JSON


```
{
    "event_id": "evt_001",
    "user_id": "user_123",
    "message": "I want to know the price"
}
```
 - 点击 **Execute**, 检查下方的 Server response 是否返回 200 及包含 **sales** 代理的回复。
4. 测试数据读取 (**GET /sessions**):
- 展开 **GET /sessions**, 点击 **Try it out**, 点击 **Execute**。
 - 检查 Server response, 确认刚才的会话数据已生成 ID 并保存。

步骤四: 准备交付文件

在项目文件夹中, 新建以下两个文件, 完成最终的打包准备。

1. 新建 **.env.example**

Code snippet

```
# Example environment variables required for the project
SECRET_TOKEN=my_secret_token_123
```

2. 新建 **README.md**

Markdown

```
# WhatsApp Chat Backend Simulation
```

```
## Run Instructions
```

```
1. Install requirements: `pip install fastapi uvicorn sqlalchemy pydantic`
```

```
2. Run server: `uvicorn main:app --reload`
```

```
3. Access API Documentation at `http://127.0.0.1:8000/docs` to test the endpoints.
```

```
## Authentication
```

```
The `POST /webhook/whatsapp` endpoint is protected.
```

```
Include header: `Authorization: Bearer my_secret_token_123`
```

步骤五: 面试现场修改预演 (Live Code Change)

考官题目: “新增一个处理财务的代理, 关键词为 'invoice', 'pay', 'billing', 回复内容为 'Let me check your account balance.'”

应对方案: 在 `main.py` 中修改以下两个函数并保存, FastAPI 会自动热重载, 你可立即去浏览器测试。

Python

1. 修改路由函数, 增加 elif

```
def route_agent(message: str) -> str:
    msg_lower = message.lower()
    if any(kw in msg_lower for kw in ["buy", "price", "product"]): return "sales"
    if any(kw in msg_lower for kw in ["problem", "issue", "complaint"]): return "support"
    if any(kw in msg_lower for kw in ["invoice", "pay", "billing"]): return "billing" # 新增此行
    return "general"
```

2. 修改回复函数, 在字典里加一条

```
def generate_reply(agent: str) -> str:
    replies = {
        "sales": "Thanks for your interest. Can you share what you are looking for?",
        "support": "Sorry you are facing this issue. Can you describe it a bit more?",
        "billing": "Let me check your account balance.", # 新增此行
        "general": "Happy to help. What would you like to know?"
    }
    return replies.get(agent, replies["general"])
```